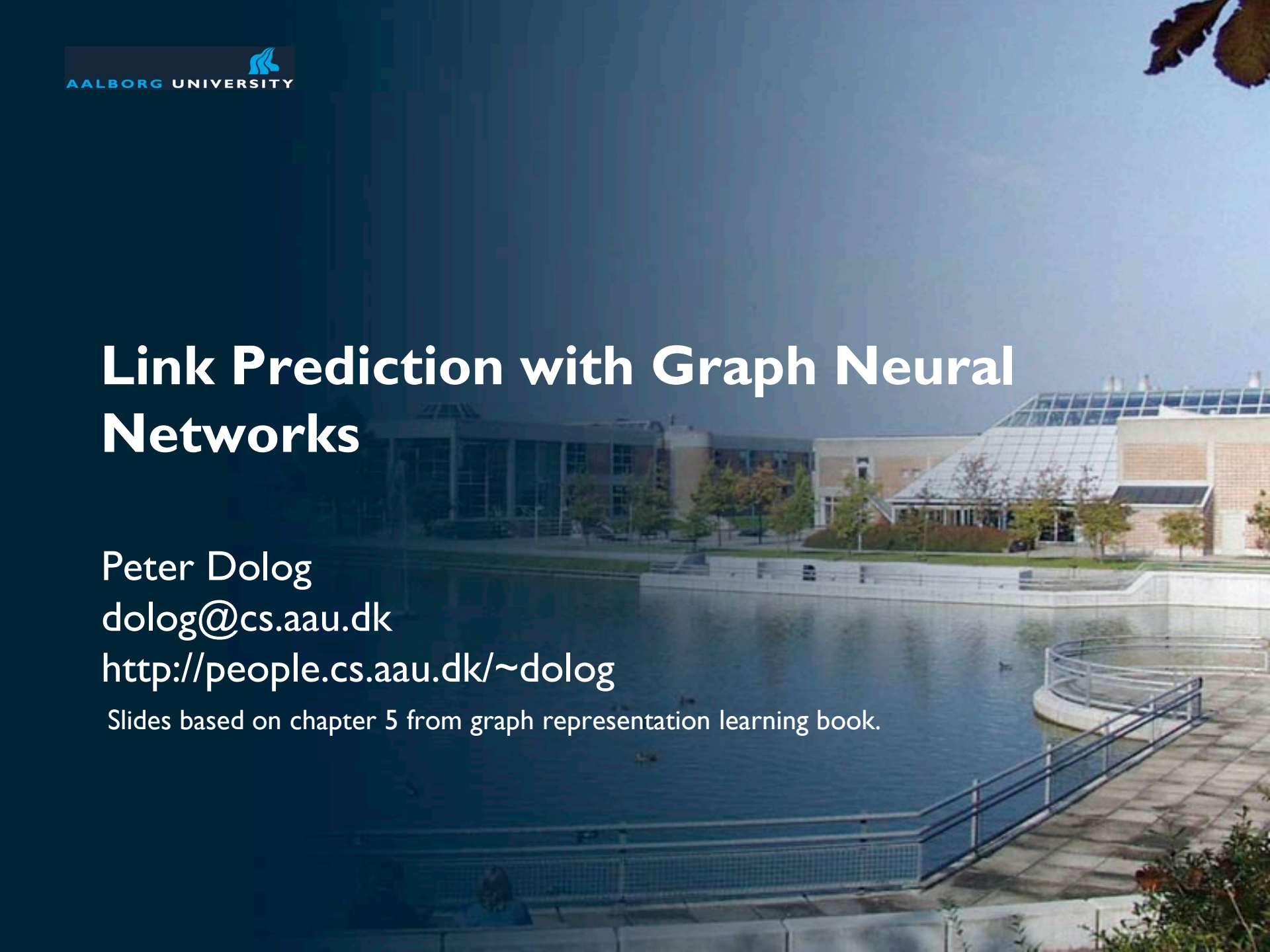


AALBORG UNIVERSITY

Link Prediction with Graph Neural Networks

Peter Dolog
dolog@cs.aau.dk
http://people.cs.aau.dk/~dolog

Slides based on chapter 5 from graph representation learning book.

Outline

Problems with previous methods
Message Passing
GNN based on Simple Message Passing
Generalized Aggregation
Generalized Update
Conclusions

Link Prediction

If the nodes are (highly) **SIMILAR**, there should be a link between them

We are talking about learning similarity from a graph

Task: Define a **similarity measure S(v;w)** between nodes based on graph structure

Last times:

- L8: local structural similarity measures and SimRank (global, based on random walk), No learning
- L9: learning node embeddings and using a similarity measure on the learned space – dot, Euclidean, dot with random walk

Problems

L8:

- Computing pairwise similarities and complete random walks can be computationally difficult for large graphs and all nodes
- Spamming is a problem
- Simple changes can affect the results
- Coverage especially for local measures poor
- …

L9:

- There are no shared parameters between nodes
- No node features used in encoding
- They know only nodes which have been there during training

Beyond Shallow Node Embeddings

Replacing “shallow encoders” by more general encoders
Using both, graph structure and graph attributes/features
Using Neural Network

Encoder/Decoder

Encoding => Learning
Decoding => Prediction
For many prediction tasks incl. link prediction

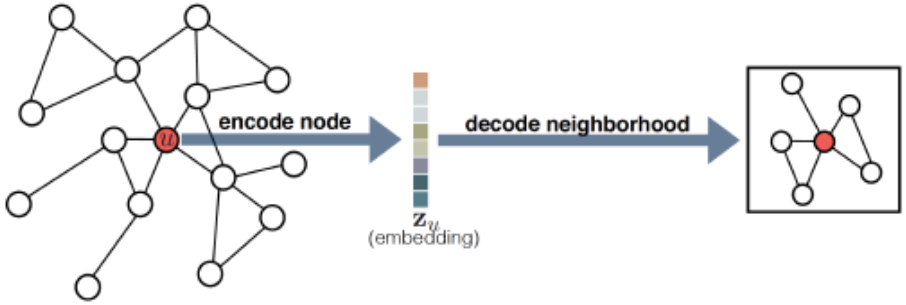


Figure 3.2: Overview of the encoder-decoder approach. The encoder maps the node u to a low-dimensional embedding $\mathbf{z}_u$. The decoder then uses $\mathbf{z}_u$ to reconstruct u's local neighborhood information.

(C) William L. Hamilton: Graph Representation Learning

General Idea

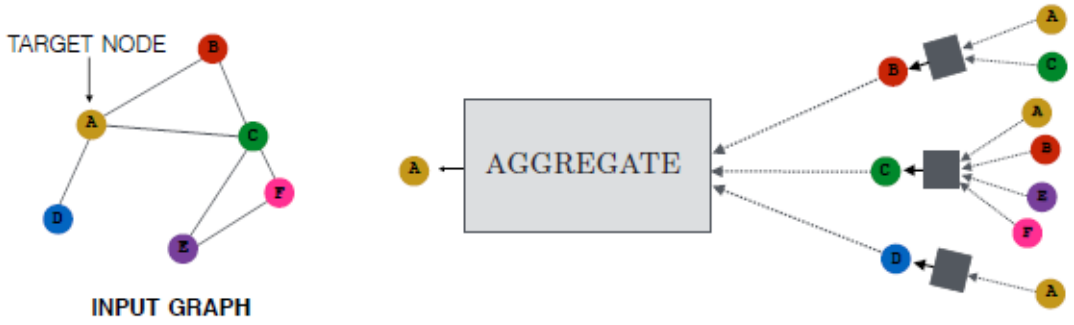
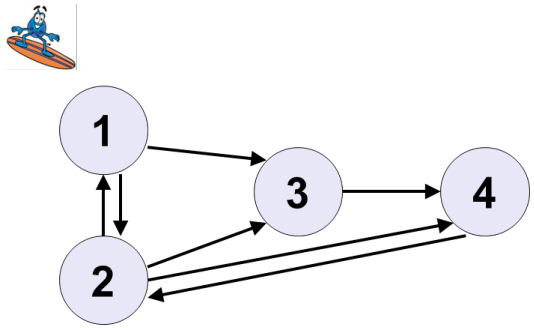
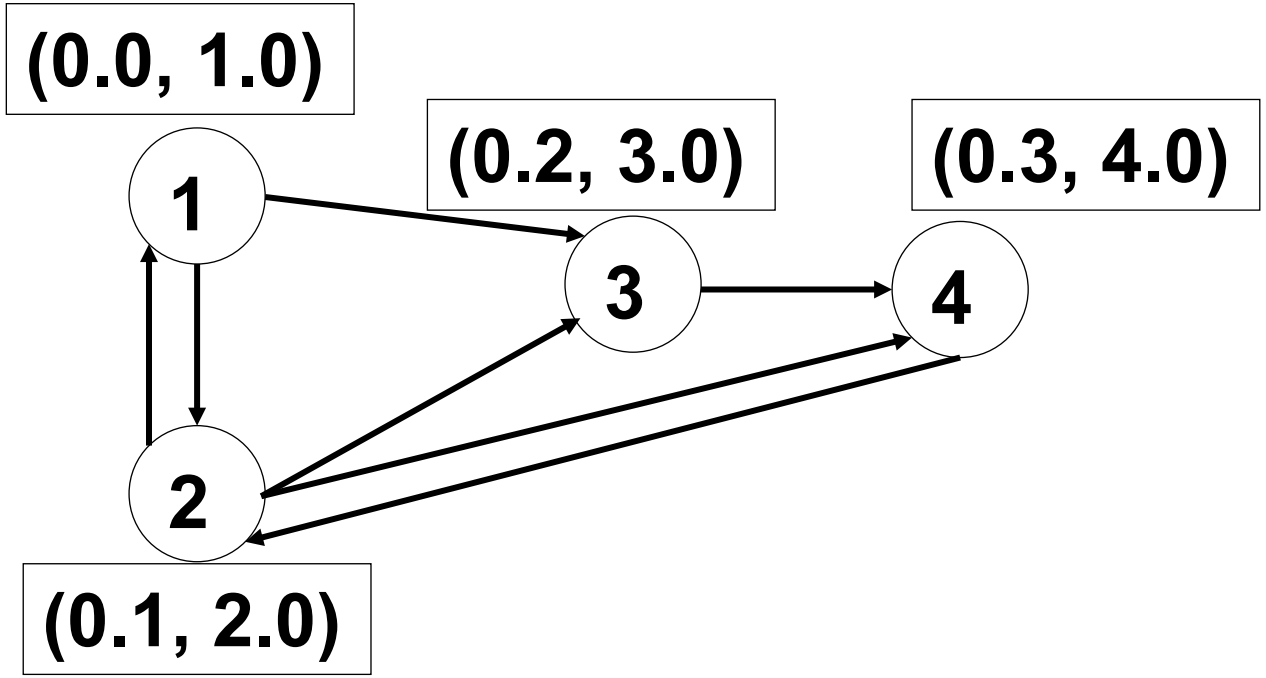


Figure 5.1: Overview of how a single node aggregates messages from its local neighborhood. The model aggregates messages from A's local graph neighbors (i.e., B, C, and D), and in turn, the messages coming from these neighbors are based on information aggregated from their respective neighborhoods, and so on. This visualization shows a two-layer version of a message-passing model. Notice that the computation graph of the GNN forms a tree structure by unfolding the neighborhood around the target node.

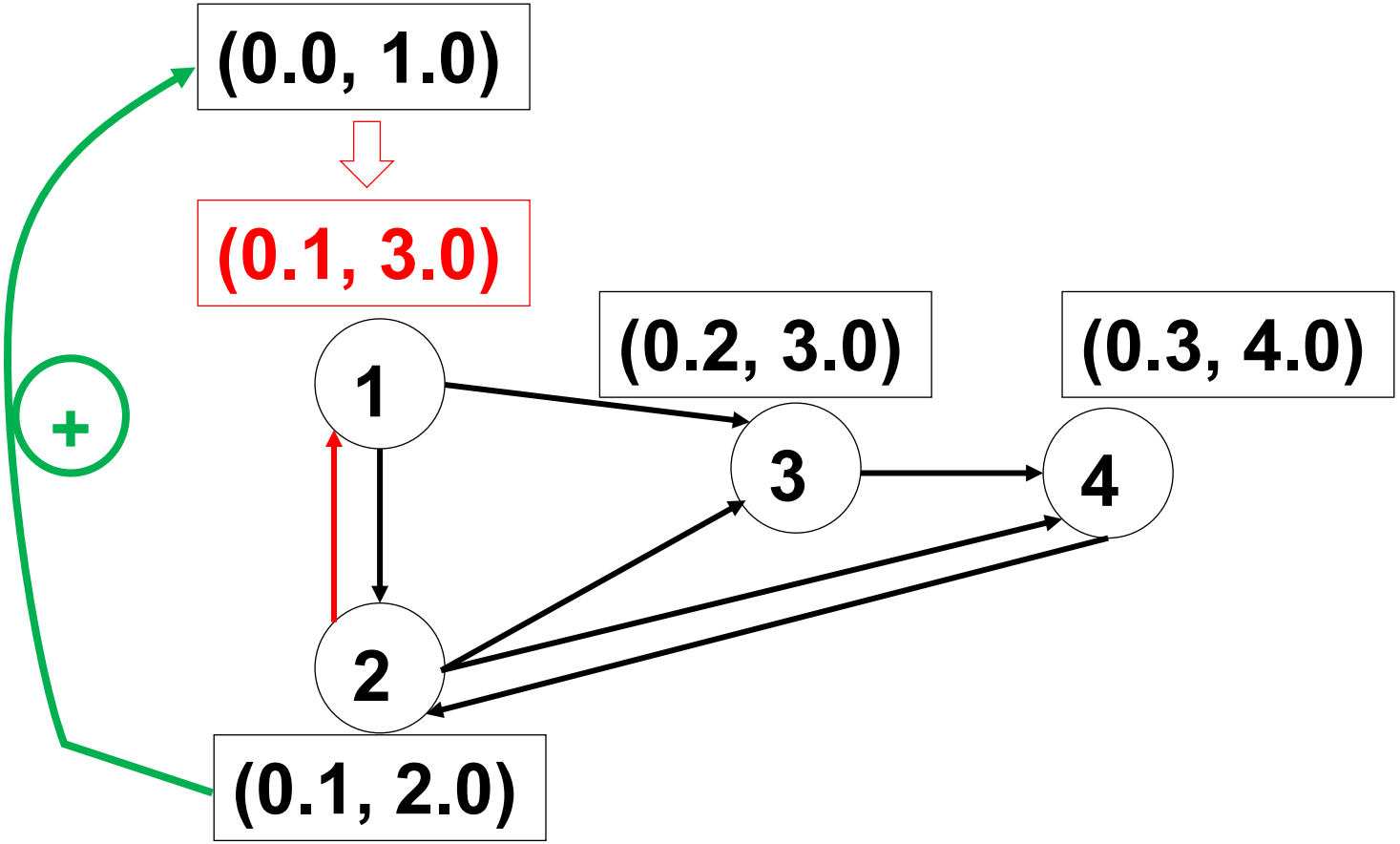
(C) William L. Hamilton: Graph Representation Learning

Illustration

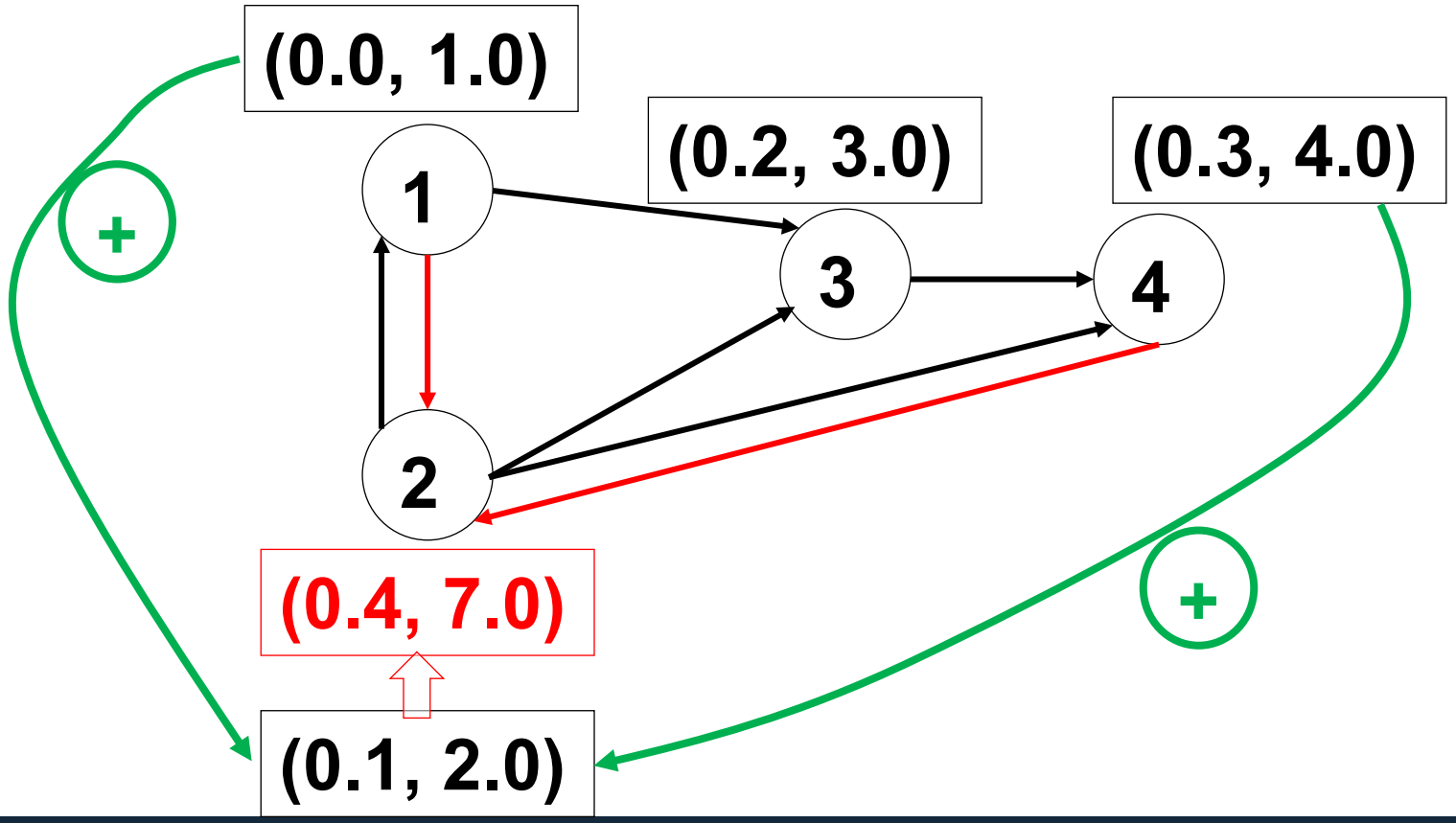




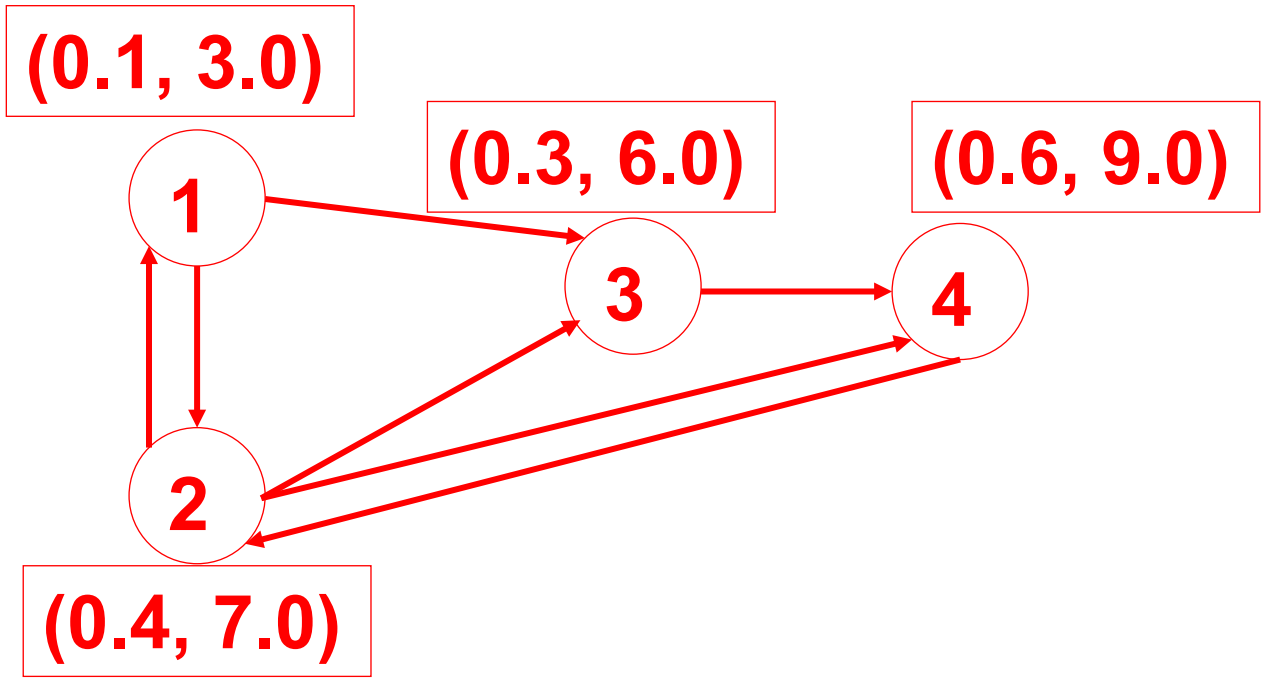
Illustration



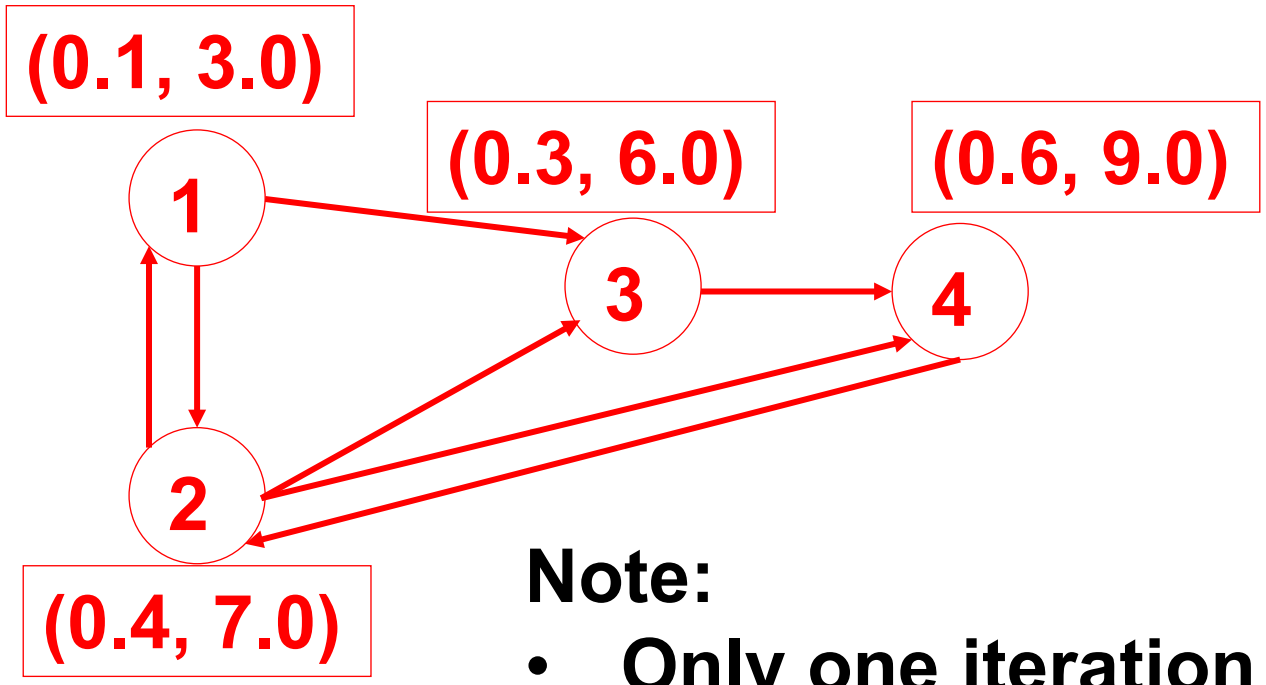
Illustration



Illustration



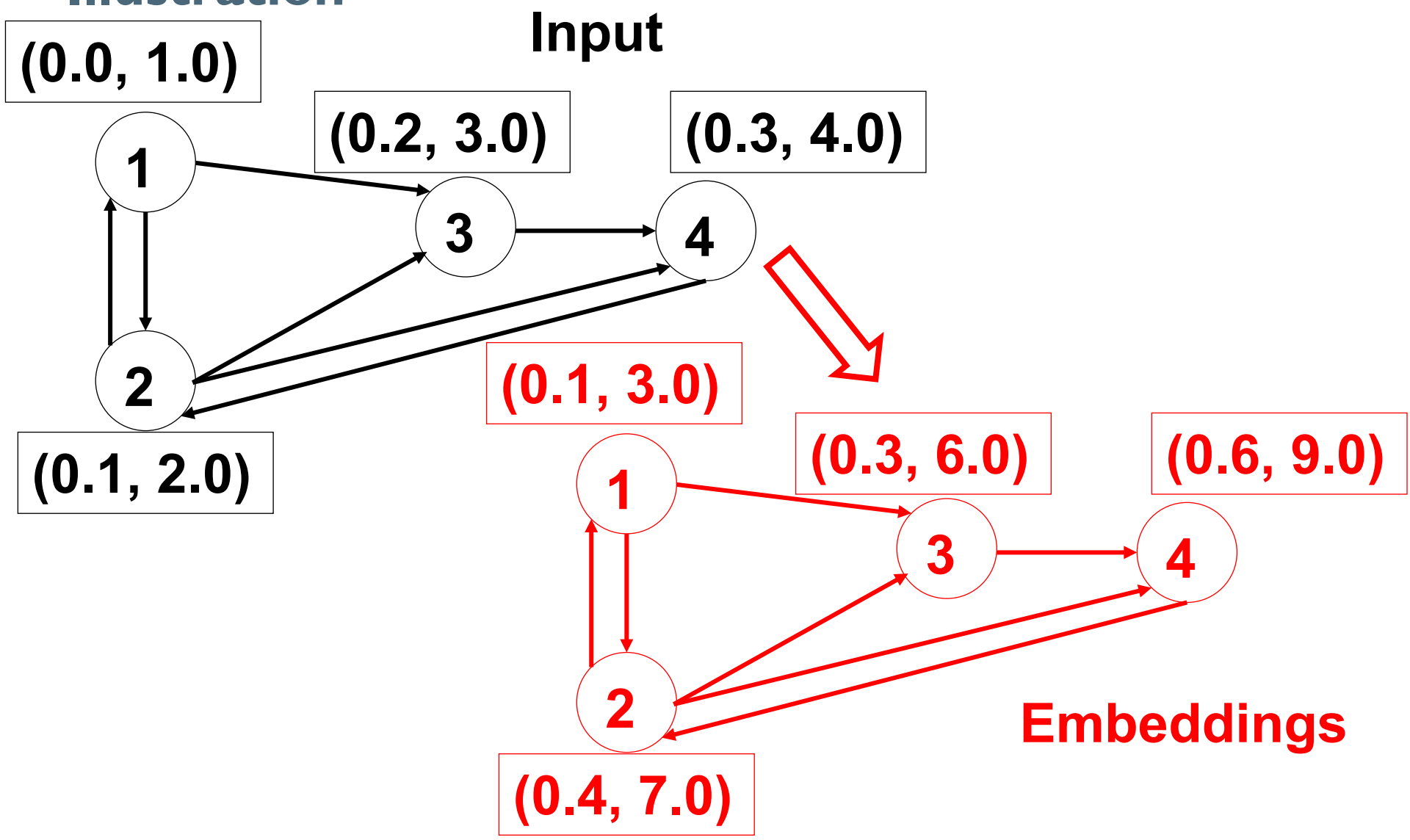
Illustration



Note:

- **Only one iteration**
- **Only local neighborhood**

Illustration



General Formalization

$$\mathbf{h}_u^{(k+1)} = UPDATE^{(k)}\left(\mathbf{h}_u^{(k)}, AGGREGATE^{(k)}\left(\left\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\right\}\right)\right)$$
$$= UPDATE^{(k)}(\mathbf{h}_u^{(k)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)})$$

$UPDATE$ and $AGGREGATE$ can be arbitrary differentiable functions

$\mathbf{m}_{\mathcal{N}(u)}$ is a "message" aggregated from u's neighborghood

$\mathbf{h}_u^{(0)}$ are set of input features for all the nodes, $\mathbf{h}_u^{(0)} = \mathrm{x}_u, \forall u \in \mathcal{V}$

$$ENC(u) = \mathbf{Z}[u] = \mathbf{h}_u^{(K)}$$

K is a number of iterations, after which we consider output as a node embeddings

Options for Node Features (Input)

- **Features from a dataset** describing a node
- **Node statistics** – node degree, node prestige, …
- **Identity features** – one hot encoding of node features – makes the model transudative though – you can consider this as a vector with dimensions for all nodes with 1 where the node identity is

General Idea

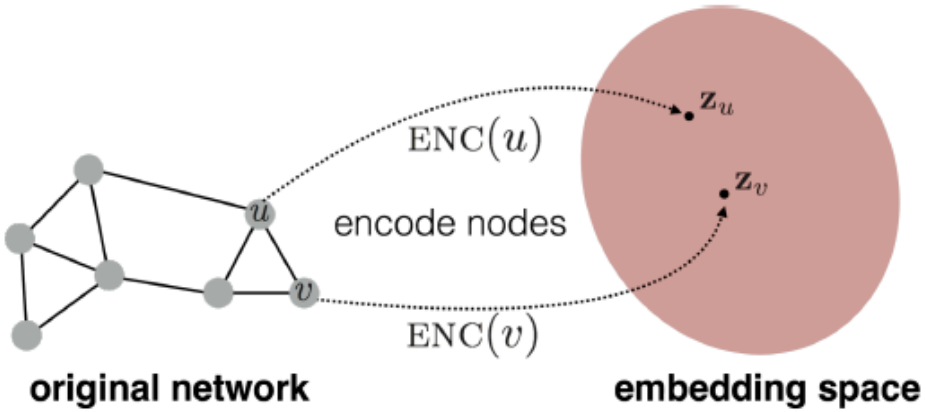


Figure 3.1: Illustration of the node embedding problem. Our goal is to learn an encoder (ENC), which maps nodes to a low-dimensional embedding space. These embeddings are optimized so that distances in the embedding space reflect the relative positions of the nodes in the original graph.

(C) William L. Hamilton: Graph Representation Learning

General Idea

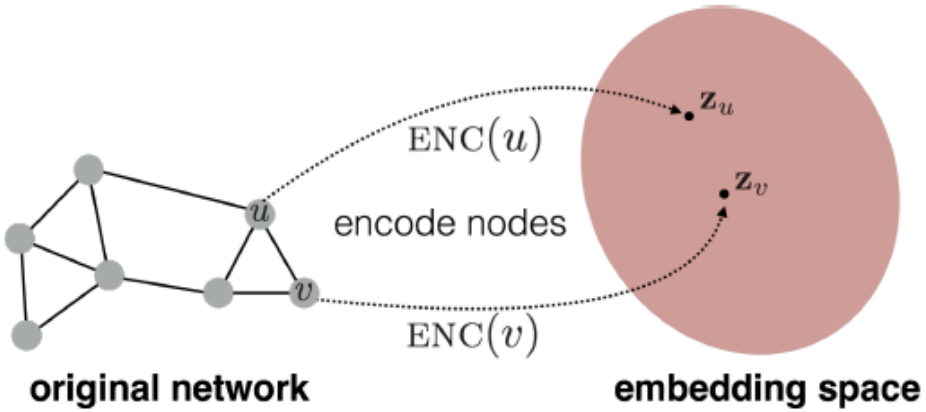


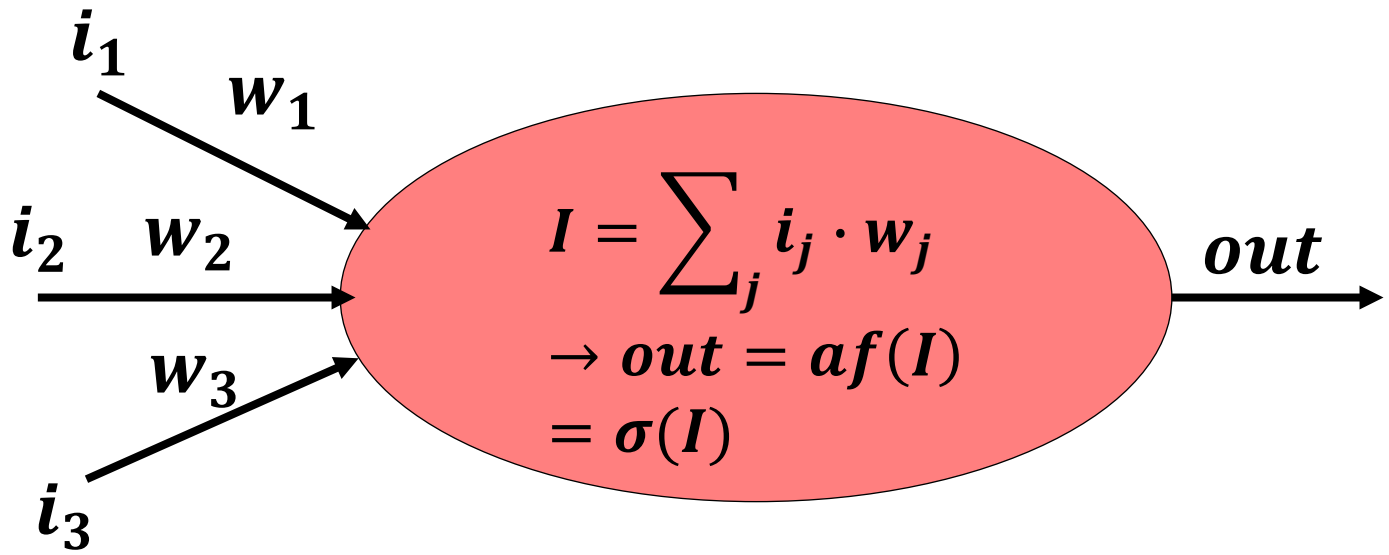
Figure 3.1: Illustration of the node embedding problem. Our goal is to learn an encoder (ENC), which maps nodes to a low-dimensional embedding space. These embeddings are optimized so that distances in the embedding space reflect the relative positions of the nodes in the original graph.

(C) William L. Hamilton: Graph Representation Learning

Still following the same idea of embeddings
But now also with node features and aggregations

Recap on Neural Networks – L4

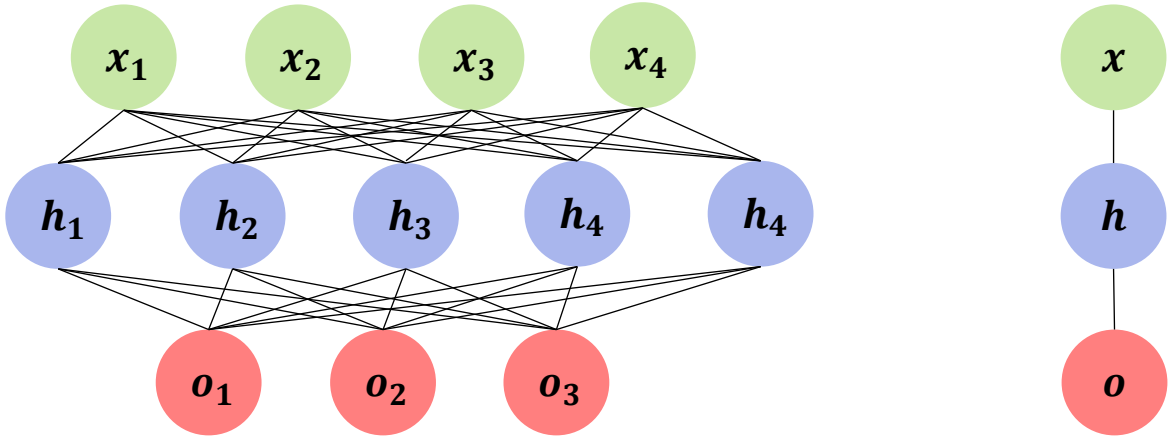
Basic Building Unit



$\sigma(I)$ with examples of Sigmoid, Tanh or ReLU

Recap on Neural Networks – L4 cnt.

NN is a real valued composite function



$$f: \mathbb{R}^4 \rightarrow \mathbb{R}^3.$$

$$f(x) = o$$

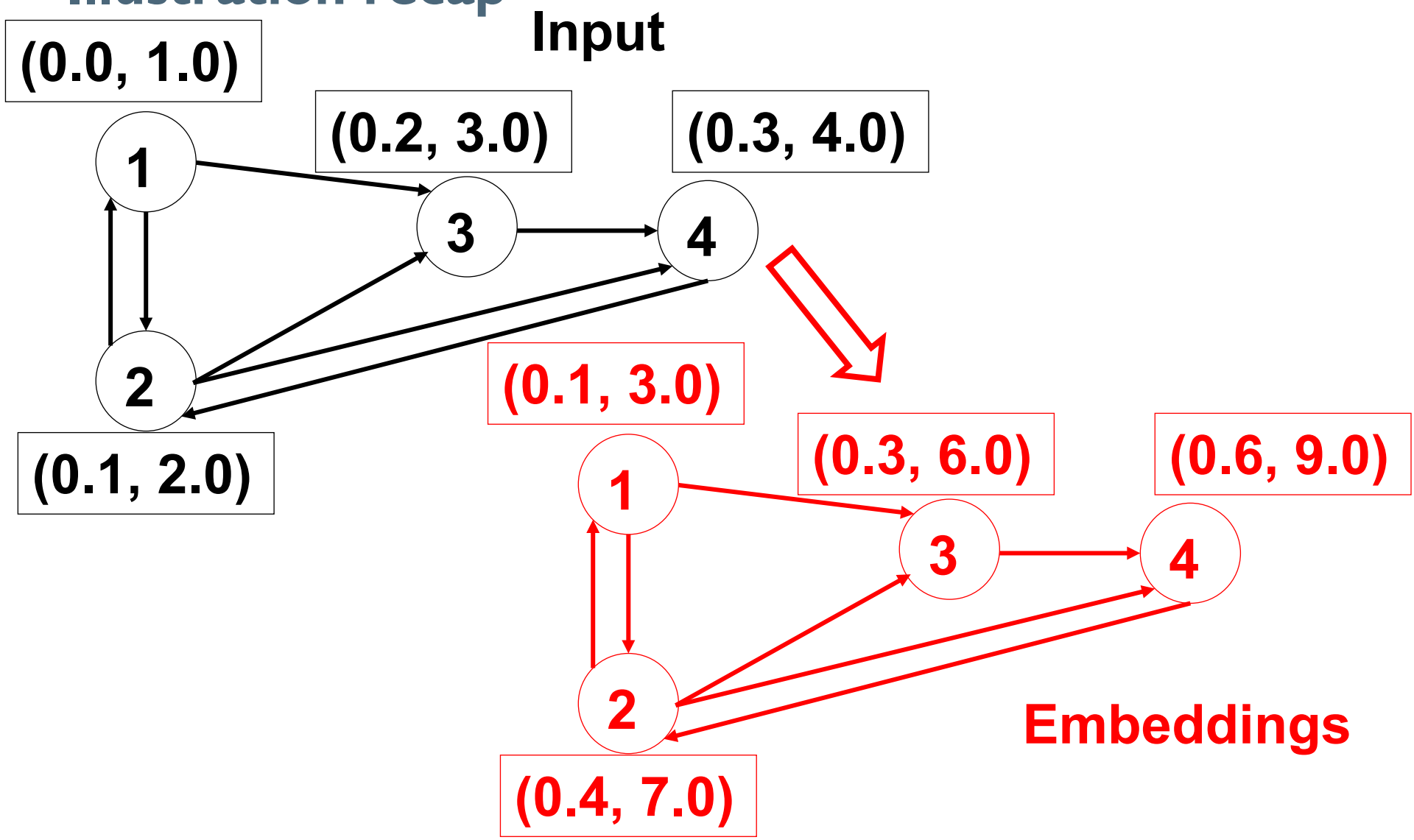
$$\mathbf{o} = f_2(\mathbf{W}_2^T \mathbf{h} + \mathbf{b}_2)$$

$$\mathbf{h} = f_1(\mathbf{W}_1^T \mathbf{x} + b_1)$$

Forward pass: prediction in one step for learning or a task

Backpropagation: update with gradient / learning

Illustration recap



Basic GNN

$$\mathbf{h}_u^{(k+1)} = \sigma(\mathbf{W}_{self}^{(k)}\mathbf{h}_u^{(k-1)} + \mathbf{W}_{neigh}^{(k)} \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)})$$

$\mathbf{W}_{self}^{(k)}, \mathbf{W}_{neigh}^{(k)} \in \mathbb{R}^{d^{(k)} \times d^{(k-1)}}$ … trainable parameter matrices

σ … an element wise nonlinearity (sigmoid, tanh, ReLU,…)

$\mathbf{b}^{(k)} \in \mathbb{R}^{d^{(k)}}$ … bias term, sometimes ommitted for notational simplicity

Analogous to MLP – linear operations followed by an elementwise nonlinearity

Basic GNN defined through UPDATE and AGGREGATE

$$AGGREGATE^{(k)}\left(\left\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\right\}\right) = \mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v$$

$$UPDATE\left(h_u, m_{\mathcal{N}(u)}\right) = \sigma\left(\mathbf{W}_{self}\mathbf{h}_u + \mathbf{W}_{neigh}\mathbf{m}_{\mathcal{N}(u)}\right)$$

Note: we have defined it at the node level, but can be defined at the graph level

Self Loops

$$\mathbf{h}_u^{(k+1)} = \sigma(\mathbf{W}_{self}^{(k)}\mathbf{h}_u^{(k-1)} + \mathbf{W}_{neigh}^{(k)} \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)})$$

$$h_u^{(k)} = AGGREGATE\left(\left\{\mathbf{h}_v^{(k-1)}, \forall v \in \mathcal{N}(u) \cup \{u\}\right\}\right)$$

i.e. instead of adding the update, we add the loops into the input graph

Equivalent to (now in graph level form):

$$\mathbf{H}^{(t)} = \sigma((\mathbf{A} + \mathbf{I})\mathbf{H}^{(t-1)}\boldsymbol{W}^{(t)})$$

Illustration

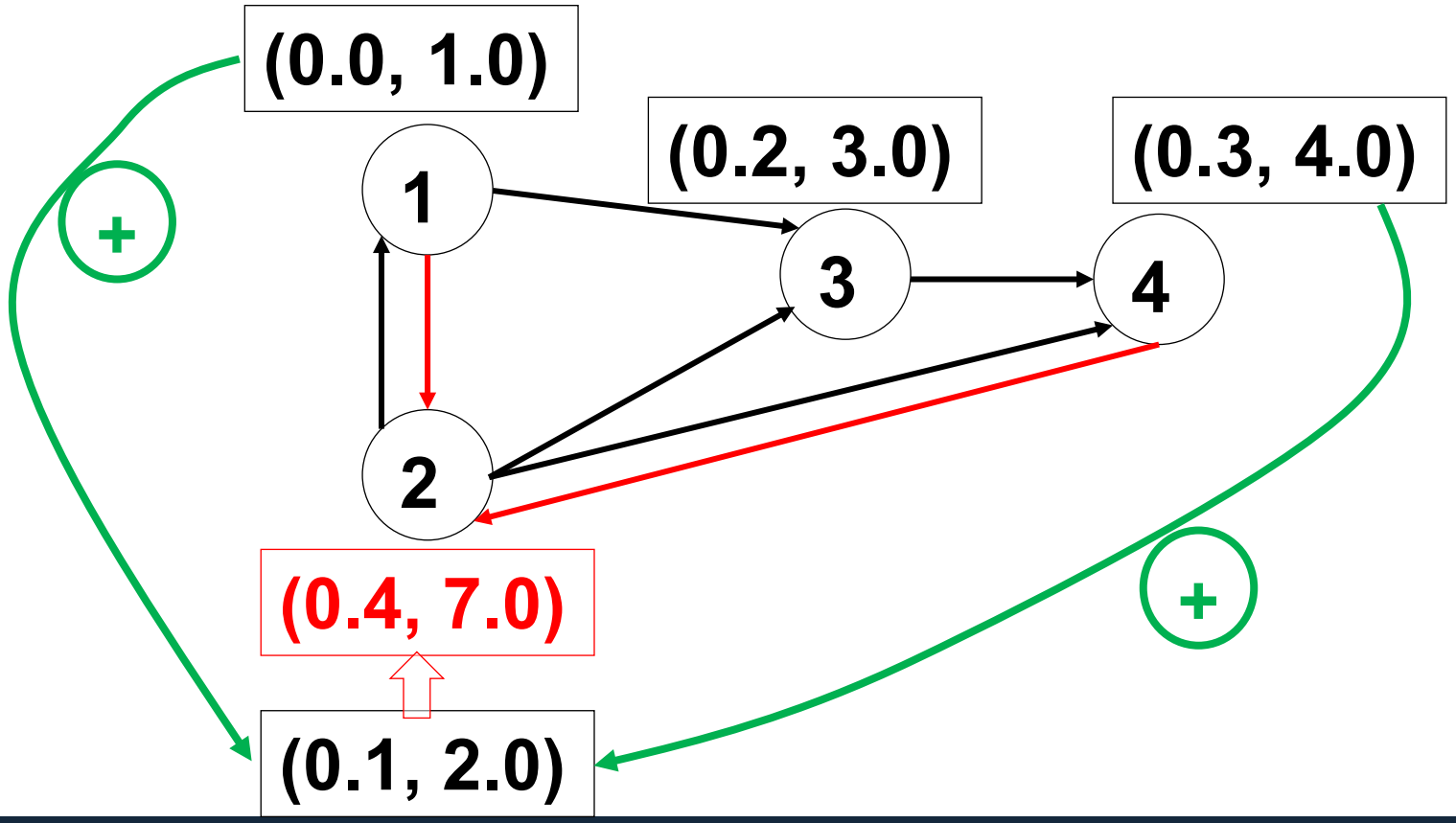


Illustration – adding neighborhood

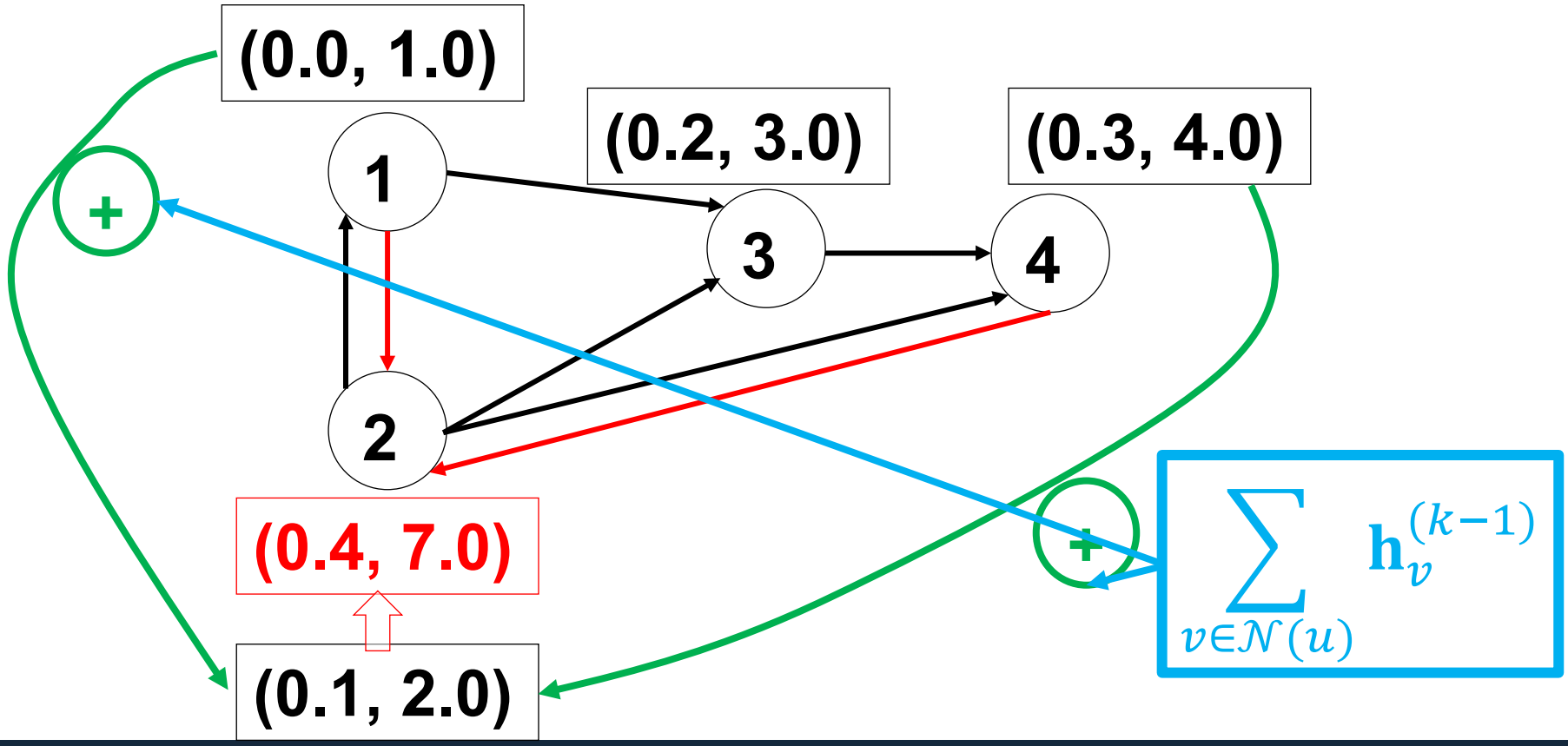


Illustration – adding neighborhood, adding selfloop

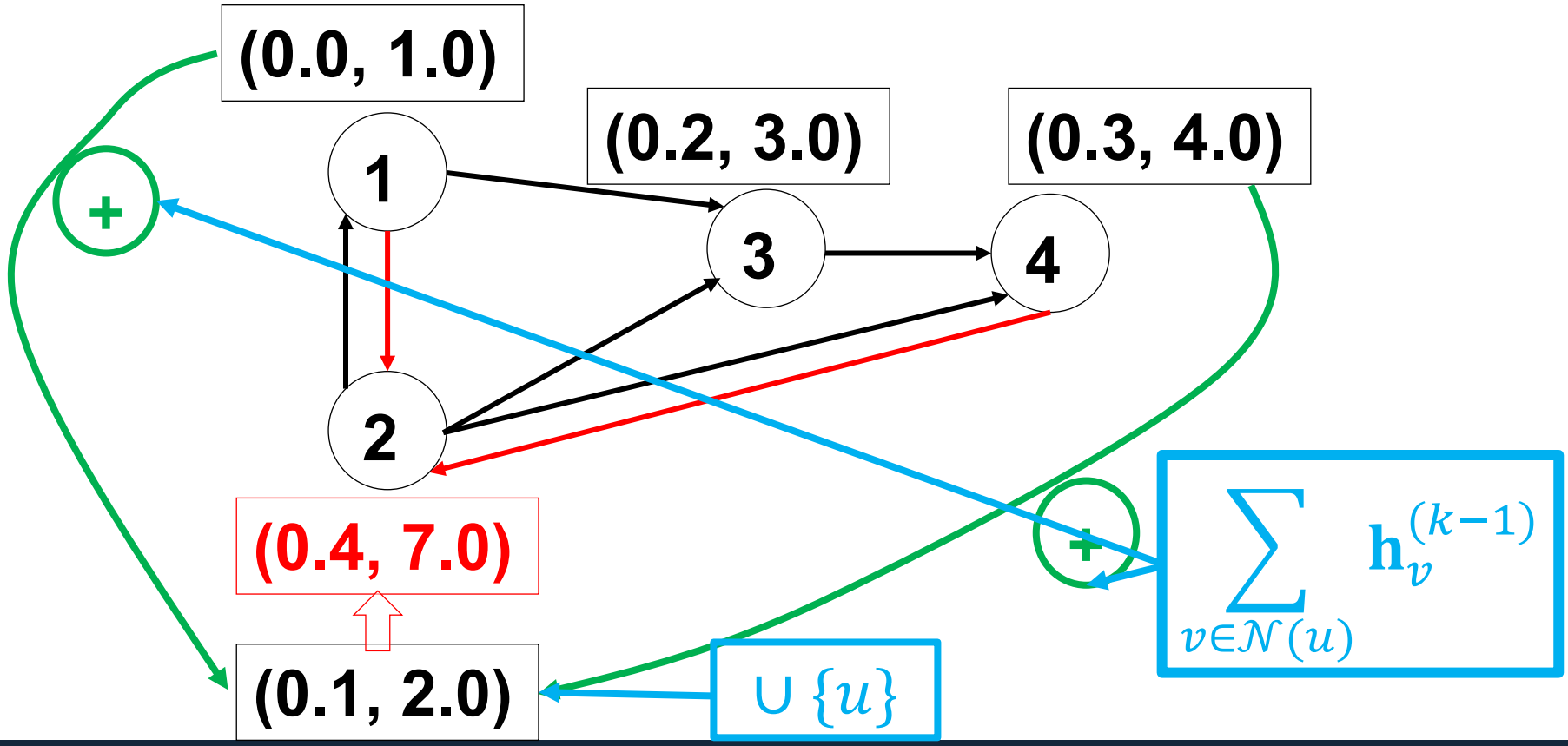


Illustration – adding neighborhood, adding selfloop, no linear transformation, no nonlinearity

$$\mathbf{H}^{(t)} = \sigma((\mathbf{A}+\mathbf{I})\mathbf{H}^{(t-1)}\mathbf{W}^{(t)})$$

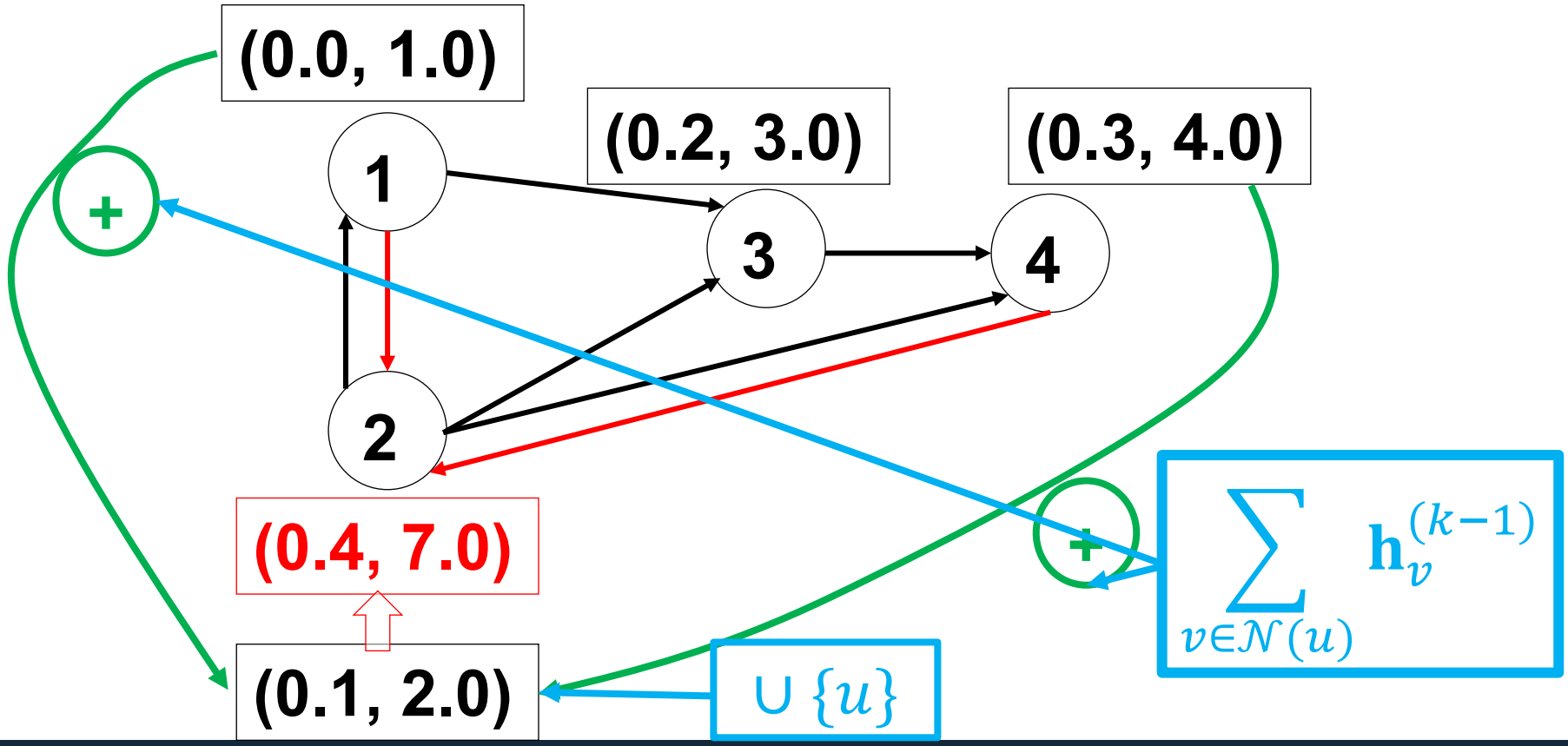


Illustration – adding neighborhood, adding selfloop, no linear transofmation, no nonlinearity

$$\mathbf{H}^{(t)} = \sigma((\mathbf{A}+\mathbf{I})\mathbf{H}^{(t-1)}\mathbf{I}\mathbf{W}^{(t)})$$

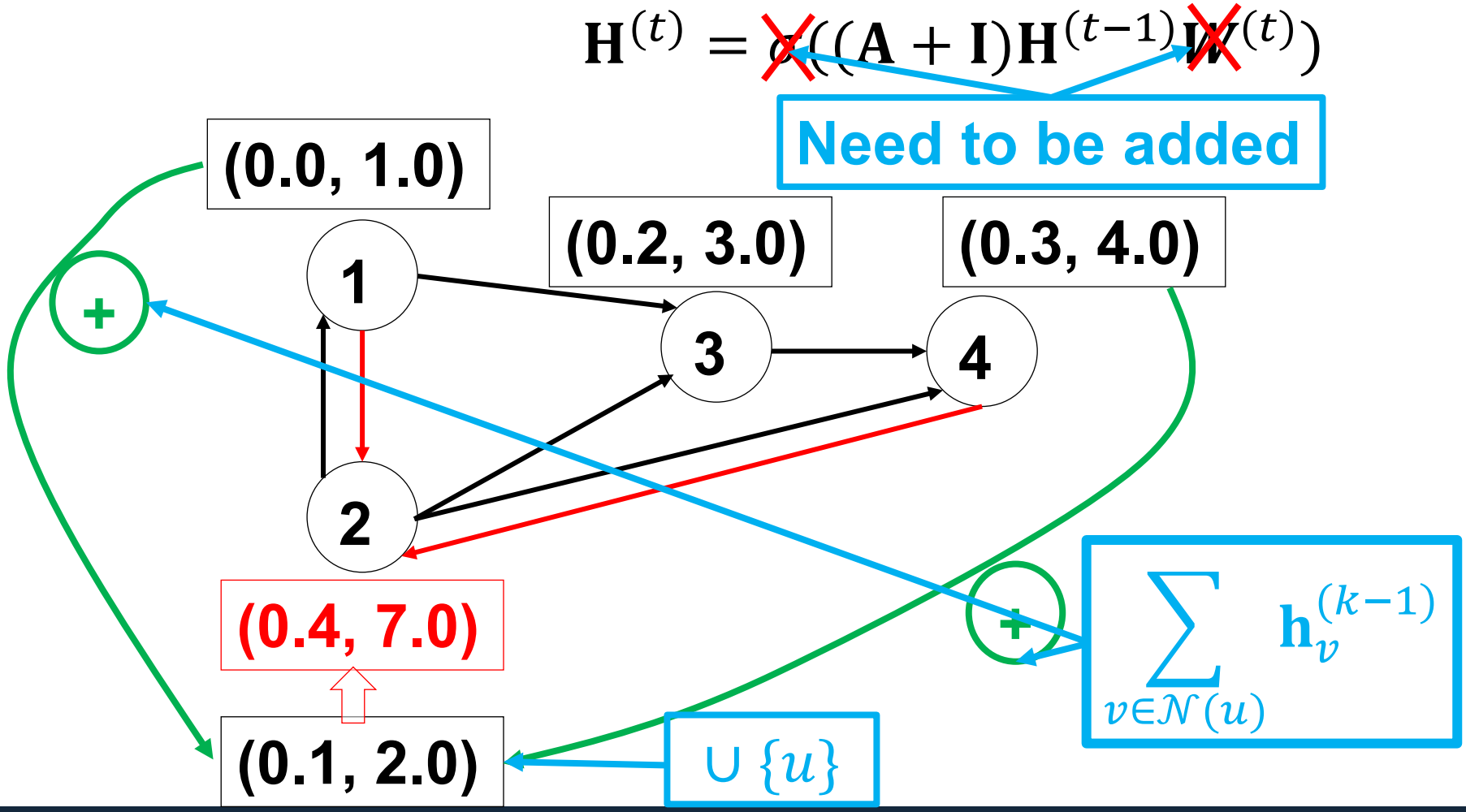


Illustration – adding neighborhood, adding selfloop, with LIN and NON LIN

$$\mathbf{H}^{(t)} = \sigma((\mathbf{A}+\mathbf{I})\mathbf{H}^{(t-1)}\boldsymbol{W}^{(t)})$$

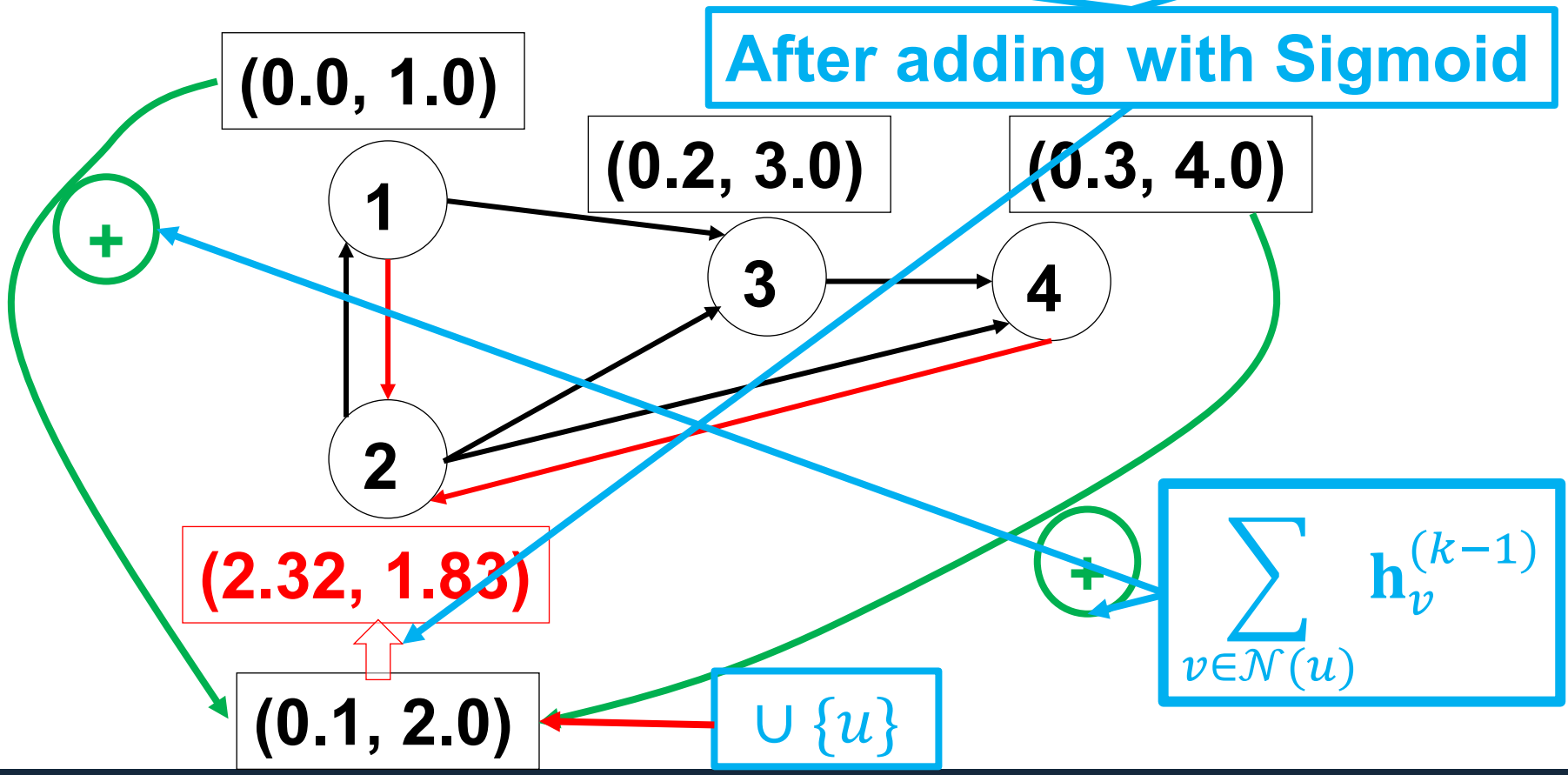
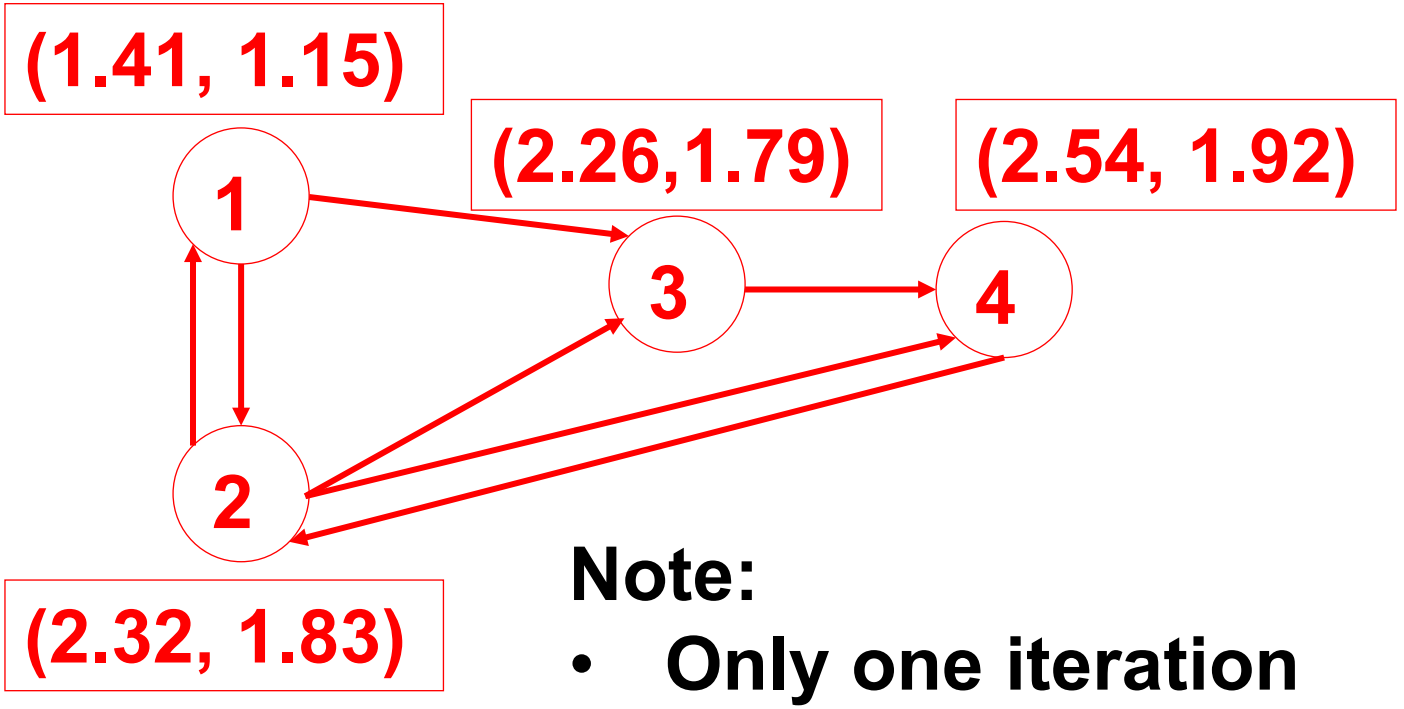


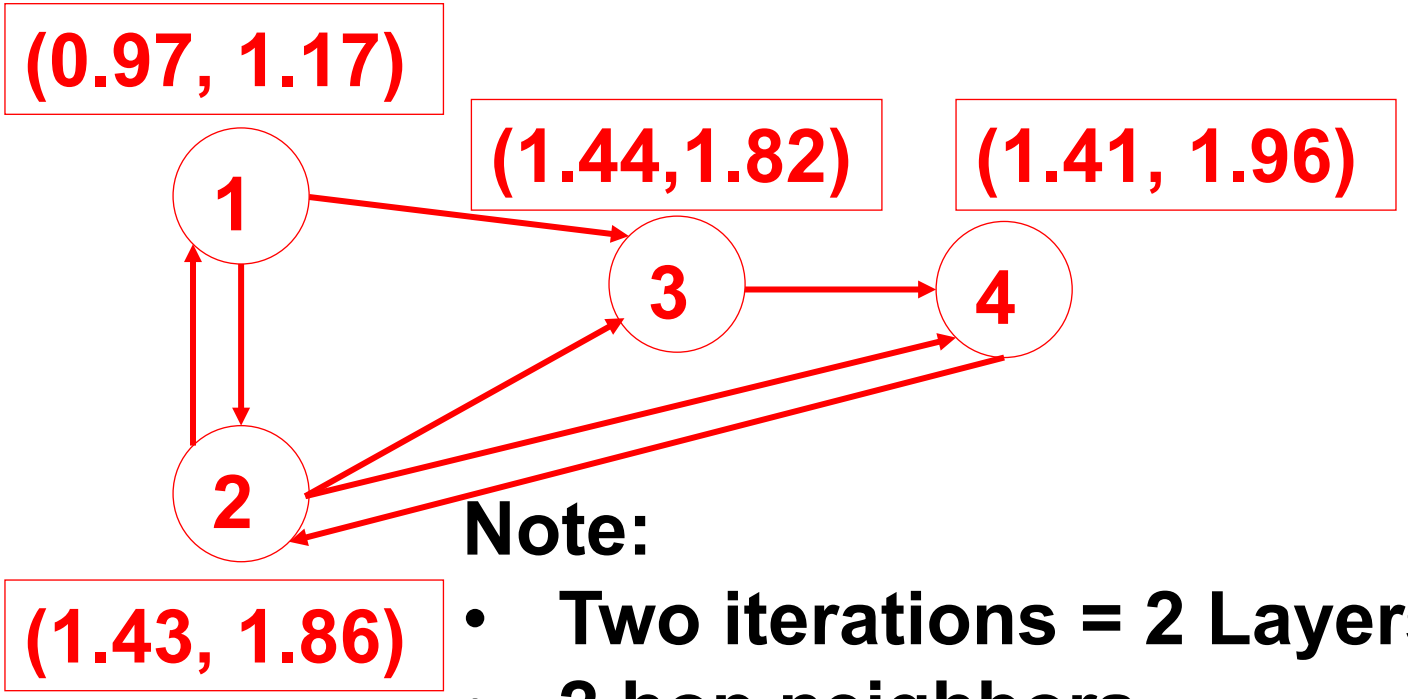
Illustration – adding N(u), selfloops, linear transformation, and Sigmoid



Note:

- **Only one iteration**
- **Only local neighborhood**

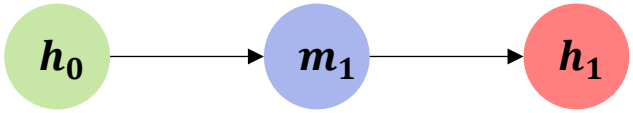
Illustration – adding N(u), selfloops, linear transformation, and Sigmoid



Note:

- **Two iterations = 2 Layers**
- **2 hop neighbors**
- **No backpropagation/learning**

Neural Network View – 1 layer



$$\boldsymbol{h_0} = \mathrm{x}_u, \forall u \in \mathcal{V}$$

$$\mathbf{m}_1 = \sum_{v \in \mathcal{N}(u)} \mathbf{h}_0$$

$$\mathbf{h_1} = \sigma(\mathbf{W}_{self}\mathbf{h}_0 + \mathbf{W}_{neigh}\mathbf{m}_1)$$

Can be combined to multiple layers

Multiple Layers of Message Passing

After passing it encodes:

- k-hop neighborhood of the graph
- Information about features in their k-hop neighborhood
- This is similar to convolutions in CNNs

Prediction

Similar as in shallow embeddings

After passing through message passing layers, we have node embeddings

To predict a link, we need a similarity

Similarity is proportional to the dot product between node embedding vectors after k's layer

$$DEC\left(\mathbf{h}_u^{(k)}, \mathbf{h}_v^{(k)}\right) = \mathbf{h}_u^{(k)T}\mathbf{h}_v^{(k)}$$

Extensions of basic GNN

Generalized Neighborhood Aggregation

- To address stability and sensitivity to node degrees
- Going beyond Sum as an aggregation
- Dealing with different importance of different neighbors

Generalized Update Methods

- To deal with over–smoothing and neighborhood influence

Neighborhood Normalization

There can be a drastic differences in sums if the degrees of nodes differ a lot

Numerical instability and problems with optimization

Instead of a Sum:

- We take an Average:

$$\mathbf{m}_{\mathcal{N}(u)} = \frac{\sum_{v \in \mathcal{N}(u)} \mathbf{h}_v}{|\mathcal{N}(u)|}$$

- Or symmetric normalization:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}}$$

Neighborhood Normalization

There can be a drastic differences in sums if the degrees of nodes differ a lot

Numerical instability and problems with optimization

Instead of a Sum:

- We take an Average:

$$\mathbf{m}_{\mathcal{N}(u)} = \frac{\sum_{v \in \mathcal{N}(u)} \mathbf{h}_v}{|\mathcal{N}(u)|}$$

Used in Graph Convolutional Networks

- Or symmetric normalization:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}}$$

To Be or Not To Be

Normalize or Not To Normalize?

This question is application specific

Normalization is most helpful when node features are far more imporant than graph structural infomation

Normalization makes differences between node degrees smaller

Normalization can be also usefull when there is a wide range of node degrees

Set Aggregators

Neighborhood aggregation is a SET function

It maps a set of neighbors embedding vectors to one aggregated embeddings vector

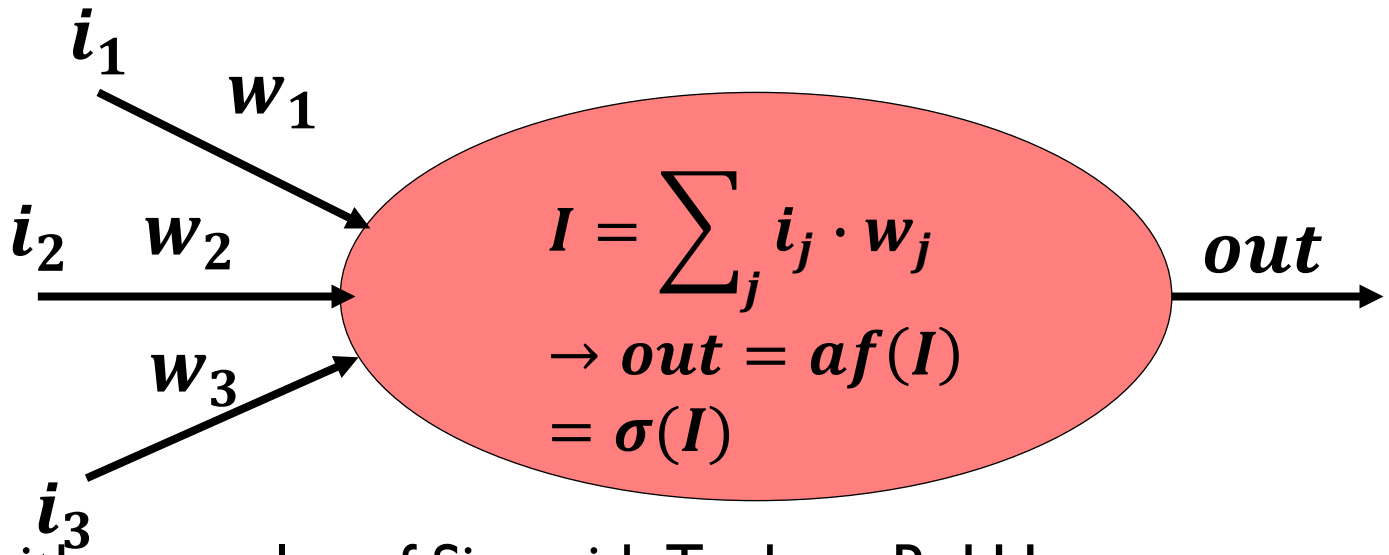
Set pooling as *a universal set function approximator*:

$$\mathbf{m}_{\mathcal{N}(u)} = \mathbf{MLP}_{\theta}\left(\sum_{v \in \mathcal{N}(u)} \mathbf{MLP}_{\phi}(\mathbf{h}_v)\right)$$

MLP

MLP – Multi Layer Perceptros

Basic Building Unit - Perceptron



$\sigma(I)$ with examples of Sigmoid, Tanh or ReLU

MLP is a NN with number of Perceptrons stacked together as multiple layers

Set Aggregators

Neighborhood aggregation is a SET function

It maps a set of neighbors embedding vectors to one aggregated embeddings vector

Set pooling as *a universal set function approximator*:

$$\mathbf{m}_{\mathcal{N}(u)} = \mathbf{MLP}_{\theta}\left(\sum_{v \in \mathcal{N}(u)} \mathbf{MLP}_{\phi}(\mathbf{h}_v)\right)$$

Learnable Parameters

Set Aggregators – Set Pooling

Neighborhood aggregation is a SET function

It maps a set of neighbors embedding vectors to one aggregated embeddings vector

Set pooling as *a universal set function approximator*:

$$\mathbf{m}_{\mathcal{N}(u)} = \mathbf{MLP}_{\theta}\left(\sum_{v \in \mathcal{N}(u)} \mathbf{MLP}_{\phi}(\mathbf{h}_v)\right)$$

Learnable Parameters

Can be replaced by other reductions: min, max

In principle, it adds extra MLP layers on top of the basic aggregations

Set Aggregators – Janossy Pooling

Based on permutation-invariant
Averages results of many possible permitations
More computationaly intensive – since we need to do a lot of permutations
In practice:

- Sample random subset of permutations
- Uses canonical ordering by node degree for example

$$\mathbf{m}_{\mathcal{N}(u)} = \mathbf{MLP}_\theta\left(\frac{1}{|\Pi|}\sum_{\pi\in\Pi}\rho_\phi(\mathbf{h}_{v_1}, \mathbf{h}_{v_2}, \ldots, \mathbf{h}_{v_{|\mathcal{N}(u)|}})_{\pi_i}\right)$$

Neighborhood Attention

Assigning an attention/importance weight for each neighbor
Weighing the influence of each neighbor in the aggregation

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v} \mathbf{h}_v$$

$$\alpha_{u,v} = \frac{exp(\mathbf{a}^T [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_v])}{\sum_{v' \in \mathcal{N}(u)} exp(\mathbf{a}^T [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_{v'}])}$$

$\mathbf{a}$ … learnable attention vector
$\mathbf{W}$ … learnable matrix
$\oplus$… concatenation

Alternative Attention Computation

$$\alpha_{u,v} = \frac{exp([\mathbf{h}_u^T \mathbf{W} \mathbf{h}_v])}{\sum_{v' \in \mathcal{N}(u)} exp([\mathbf{h}_u^T \mathbf{W} \mathbf{h}_{v'}])}$$

OR

$$\alpha_{u,v} = \frac{exp(\mathbf{MLP}(\mathbf{h}_u, \mathbf{h}_v))}{\sum_{v' \in \mathcal{N}(u)} exp(\mathbf{MLP}(\mathbf{h}_u, \mathbf{h}_{v'}))}$$

Generalized Updates – Concatenation

Over-smoothing:

- After number of updates the node specific information is "washed out"
- The aggregated embeddings dominate by the message from neighbors

Keeping previous information by concatenation instead of summing up:

$$UPDATE_{concat}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) = [UPDATE_{BASE}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) \oplus \mathbf{h}_u]$$

Generalized Updates – Linear Interpolation

$$UPDATE_{interpolate}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) = \alpha_1 \circ UPDATE_{BASE}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) + \alpha_2 \circ \mathbf{h}_u$$

- $\circ$ … elementwise multiplication
- $\alpha_1, \alpha_2 \in [0,1]^d$ … gating vectors, $\alpha_2 = 1 - \alpha_1$, can be learned through a GNN, MLP, or generated

Conclusions

We can combine node information/features with graph structure
Learn more robust node representation
Based on simple message passing
With various extensions presented
More solutions in the book
Would require introduction to other forms of neural networks